AI assistant setup: instructions, context and prompt

Objectives

- Name the three inputs an assistant works from: standing instructions, shared context and your prompt
- Predict how missing context changes an assistant's proposal
- Choose a workspace scope that gives the assistant the project it needs and nothing beyond it

Teaching

Start with a guess

You ask an AI assistant: “Add a line to start.sh that prints the port the app uses.” You did not share start.sh or any project rules.

Every prompt, file and answer here is a synthetic example written for this lesson. Nothing calls a real AI model.

Three inputs, nothing else

An assistant builds its answer from three inputs: standing instructions (a project rules file it reads with every request; the file name and support vary by tool), shared context (the files or snippets it can see) and your prompt.

It has no other view of your project. A missing input gets filled with something plausible. Whatever comes back is a proposal until you check it.

Setup decides what it can see

Setup comes down to three choices: use a tool your program or employer approves, with the account they provide; open the workspace at the project folder for your task; keep a standing instructions file in that project.

Opening the project folder helps the assistant pick up the right project context. It is not a guaranteed boundary: what the assistant may read, and which files are excluded, depends on the tool and its settings. The model here is an illustrative view: too wide, and private or unrelated files come into view; too narrow, and it loses files your change depends on.

Apply it to a new request

A teammate, Jo, has a proposal that looks finished. Use the three inputs to decide the next step.

Model

What the assistant receives

Standing instructionsnot sharedAI-RULES.md

Shared contextnot shared

```
start.sh
```

Your promptsentAdd a line to start.sh that prints the port the app uses.

Proposal, not verified (synthetic)

```
?
```

Hidden until you make your prediction.

Recap

- An assistant works from three inputs: standing instructions, shared context and your prompt.
- When an input is missing, it fills the gap with a plausible guess.
- Set up with an approved tool, a workspace at the project folder and a rules file in the project.
- Every answer is a proposal until you check it.

Next topic: Rules of use: what to share and what you own

Cheat sheet

Standing instructions	project rules the tool reads with every request (file name varies by tool)
Shared context	the files or snippets the assistant can see
Prompt	your request, in your words
Workspace	open it at the project folder for the task

Sources

Claims on this page were checked against the live official pages below on 6 October 2026 (UTC). Links open outside this lesson.

- Adding repository custom instructions for GitHub Copilot: .github/copilot-instructions.md is added to requests automatically. One tool's instructions file; names and support differ by tool.
- How Claude remembers your project (Claude Code Docs): CLAUDE.md files. A second tool, a different file name.
- Excluding content from GitHub Copilot: content exclusion settings. Which files a tool may read depends on the tool and its settings; an opened folder is not a guaranteed boundary.
- Best practices for using GitHub Copilot: Check Copilot's work. It can make mistakes; validate what it suggests.
- Shell Command Language: Parameter Expansion, Use Default Values. Explains ${PORT:-8080} in the sh fixture. POSIX.1-2024 (IEEE Std 1003.1-2024), the standard behind sh; GNU Bash and coreutils manuals could not be reached on 6 October 2026, so Bash- or GNU-specific details are not separately verified.

Not tied to one source: the general picture of an assistant filling missing context with plausible guesses is teaching guidance, consistent with the validation advice above, not a documented guarantee for every tool.

Static interaction alternatives

predict

Prediction (not scored). You ask an assistant to add a line to start.sh that prints the app's port, but share neither start.sh nor any project rules. Options: a line with a made-up port such as 3000; the exact port your project uses; a question asking for start.sh; no answer. In this example it returns a line with a made-up port (echo "Using port 3000"): with the file missing, it fills the gap with a plausible value. The exact port would be luck, not knowledge; some assistants do ask for the file, but you cannot count on that; and it did return a proposal.

explore

Toggle: standing instructions (AI-RULES.md: read settings from environment variables; never hard-code ports or secrets). With instructions only, the proposal is echo "Using port $PORT": it follows the rule but cannot see start.sh, so it does not know PORT has a default or where the line goes.

explore

Toggle: shared context (start.sh, which sets PORT="${PORT:-8080}"). With context only, the proposal hard-codes echo "Using port 8080", which prints the wrong port whenever PORT is set and breaks the rules file. With neither input it invents port 3000. With both, it proposes PORT="${PORT:-8080}" then echo "Using port $PORT": it fits the file and the rule, and it is still a proposal until someone runs it.

explore

Workspace scope for the pep project. /home/ana is too wide: notes, the shell profile and other files come into view. /home/ana/projects/pep is the right size: start.sh, AI-RULES.md and README.md are in view (so is .env, which the next topic covers). Only start.sh is too narrow: the rules file and any file start.sh depends on drop out. / (the whole computer) is far too wide: every readable folder comes into view.

mcq

Graded check with a confidence rating. Jo shares only a test error message, no test file and no rules file, and the proposal renames a function three other files call. The right next step is to share the test file and the callers, keep the rules file, then review again. Accepting because the summary says the test passes is wrong: a description is not evidence. Resending the same prompt keeps the same missing inputs. Opening the whole home folder overshares and does not clarify the task. Retry in the model: the fix is in the context (add the test file and the callers), not a longer prompt, not removing the instructions, and not treating the proposal as already verified.